

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

1 Traza de estado Interno

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    // PROBLEMA FUNDAMENTAL: Este setter permite mutar el objeto DESPUÉS de
    insertarlo en el HashSet.
    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        // El 'equals' depende del código Y del nivel de acceso.
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo, tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        // EL GRAVE PROBLEMA: El hash depende del 'nivelAcceso', que es MUTABLE.
        // Al cambiar el nivel, el hash cambia, pero el objeto ya está guardado en otro bucket.
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
    }
}
```

```

// 1. Se crea el objeto con nivelAcceso = 2.
TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

// 2. Se inserta en el HashSet.
// - Se calcula el hash con nivel=2 -> se guarda en el BUCKET "A".
tarjetasValidas.add(t1);

// 3. MUTACIÓN PELIGROSA: El nivel cambia a 5, pero el objeto sigue físicamente en
el BUCKET "A".
// - Si volvemos a calcular el hash ahora, daría el hash de nivel=5 -> apuntaría al
BUCKET "B".
t1.setNivelAcceso(5);

// 4. Búsqueda fallida:
// - contains() calcula el hash actual (nivel=5) y busca en el BUCKET "B".
// - Como el objeto está en el BUCKET "A", NO LO ENCUENTRA.
System.out.println("¿ Tarjeta válida en sistema?: " + tarjetasValidas.contains(t1)); // false
(ERROR)

// 5. El size sigue siendo 1 porque el objeto NUNCA se eliminó, pero es INVISIBLE
para el HashSet.
System.out.println("Cantidad de tarjetas registradas: " + tarjetasValidas.size()); // 1
}
}

```

2 Explicación del cajón

- **Bucket de inserción (add):** Se calcula con hashCode() original (nivel=2). Ej: índice 5.
- **Bucket de búsqueda (contains):** Se calcula con hashCode() mutado (nivel=5). Ej: índice 12.
- **Conclusión:** Java busca en el índice 12, pero el objeto reside en el 5. Nunca se encuentran.

3 Fugas de memoria

En un servidor 24/7, este patrón se repite con millones de objetos. Cada objeto mutado queda "fantasma" dentro de la tabla interna del HashSet. El Garbage Collector no puede liberarlos porque la tabla del HashSet mantiene referencias fuertes a ellos (aunque estén en el bucket equivocado). A largo plazo, el Heap se satura y lanza OutOfMemoryError.

4 Refactorización obligatoria

Corrección 1

```
import java.util.*;
```

```
class TarjetaAcceso {
    // CORRECCIÓN 1: Campos declarados 'final' para evitar la mutación.
    private final String codigo;
    private final int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    // CORRECCIÓN 2: Se ELIMINA el método 'setNivelAcceso'. No se puede modificar el
    objeto.
    // En su lugar, se provee un método que DEVUELVE UNA NUEVA INSTANCIA con el
    nivel cambiado.
    public TarjetaAcceso conNuevoNivel(int nuevoNivel) {
        // Esto crea un objeto nuevo, el anterior sigue siendo válido en el HashSet.
        return new TarjetaAcceso(this.codigo, nuevoNivel);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof TarjetaAcceso)) return false;
        TarjetaAcceso that = (TarjetaAcceso) o;
        // CORRECCIÓN 3: Ambos campos son inmutables, el hash nunca cambia.
        return nivelAcceso == that.nivelAcceso && Objects.equals(codigo, that.codigo);
    }

    @Override
    public int hashCode() {
        // CORRECCIÓN 4: Como los campos no cambian, este hash es ESTABLE para toda
        la vida del objeto.
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAccesoSeguro {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();

        // Se crea e inserta la tarjeta.
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
    }
}
```

```

tarjetasValidas.add(t1);

// CORRECCIÓN 5: Para "cambiar" el nivel, se reemplaza la referencia en el Set.
// 1. Se remueve la vieja (funciona porque el hash original sigue siendo el mismo).
// 2. Se agrega la nueva instancia.
TarjetaAcceso t1Actualizado = t1.conNuevoNivel(5);
tarjetasValidas.remove(t1);      // Elimina la instancia antigua (hash estable).
tarjetasValidas.add(t1Actualizado); // Agrega la nueva (con su nuevo hash).

// Ahora contains() funciona correctamente.
System.out.println("¿ Tarjeta válida?: " + tarjetasValidas.contains(t1Actualizado)); // true
System.out.println("Cantidad: " + tarjetasValidas.size()); // 1
}
}

```

Corrección 2

```

import java.util.*;

class TarjetaAcceso {
    // CORRECCIÓN: 'codigo' es la CLAVE ÚNICA de la entidad (immutable).
    private final String codigo;
    // 'nivelAcceso' puede mutar libremente, pero NO PARTICIPA en el hash/equals.
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    // El setter es seguro ahora, porque modificar esto NO cambia el hash.
    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof TarjetaAcceso)) return false;
        TarjetaAcceso that = (TarjetaAcceso) o;
        // CORRECCIÓN: equals solo compara el 'codigo' (que es único e immutable).
        // Dos tarjetas son la misma si tienen el mismo código.
        return Objects.equals(codigo, that.codigo);
    }

    @Override
    public int hashCode() {

```

```

        // CORRECCIÓN: El hash depende ÚNICAMENTE del código.
        // Aunque el nivel cambie, el hash permanece IDÉNTICO.
        return Objects.hash(codigo);
    }
}

public class ControlAccesoPorCodigo {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();

        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
        tarjetasValidas.add(t1);

        // Ahora es SEGURO mutar el nivel, porque el hash NO cambia.
        t1.setNivelAcceso(5);

        // contains() funciona perfectamente porque el bucket de búsqueda es el mismo.
        System.out.println("¿ Tarjeta válida?: " + tarjetasValidas.contains(t1)); // true
(CORREGIDO)
        System.out.println("Cantidad: " + tarjetasValidas.size()); // 1
    }
}

```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Requerimiento	Interfaz	Implementación	Complejidad
Auditoría de transacciones	List	ArrayList	Add():O(1) iterador():O(n)
Ofertas relámpago	NavigableSet	TreeSet	Add():O(log n) first():O(1) last():O(1)
Búsqueda de sesiones por Token	Map	HashMap	get():O(1)

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

```
import java.util.*;
```

```
public class OptimizadorCadenas {
```

```

/**
 * Método optimizado para encontrar el primer carácter no repetido en una cadena.
 * Complejidad: O(n) tiempo, O(k) espacio (k = caracteres distintos).
 *
 * @param texto Cadena de entrada.
 * @return El primer carácter con frecuencia 1, o null si no existe.
 */
public static Character primerCaracterNoRepetido(String texto) {
    // PASO 1: Mapa para contar frecuencias.
    // Usamos LinkedHashMap para preservar el orden de inserción.
    // Esto nos permite, en el paso 2, iterar en el orden en que aparecen los caracteres en
    el texto.
    Map<Character, Integer> frecuencias = new LinkedHashMap<>();

    // PRIMER RECORRIDO (O(n)):
    // Recorremos la cadena de principio a fin para contar cuántas veces aparece cada
    letra.
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        // getOrDefault: si la clave existe, devuelve su valor; si no, devuelve 0.
        // Luego sumamos 1 y lo guardamos en el mapa.
        frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);
    }

    // SEGUNDO RECORRIDO (O(n) en el peor caso, pero normalmente solo recorre el
    mapa de claves):
    // Iteramos sobre las entradas del mapa. Como es LinkedHashMap, el orden es el de
    primera aparición.
    for (Map.Entry<Character, Integer> entrada : frecuencias.entrySet()) {
        // Si la frecuencia es exactamente 1, es el primer carácter no repetido.
        if (entrada.getValue() == 1) {
            return entrada.getKey(); // Devolvemos el carácter.
        }
    }

    // Si todos los caracteres se repiten, retornamos null.
    return null;
}

// ALTERNATIVA MÁS MEMORIA EFICIENTE (usando HashMap común y segundo
recorrido sobre el texto):
public static Character primerCaracterNoRepetidoV2(String texto) {
    // Mapa sin orden (HashMap). Ocupa menos memoria que LinkedHashMap.
    Map<Character, Integer> frecuencias = new HashMap<>();

    // Primer recorrido (O(n)): conteo.
    for (char c : texto.toCharArray()) {

```

```

        frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);
    }

    // Segundo recorrido (O(n)): recorremos el texto ORIGINAL en orden.
    // Al recorrer el texto directamente, aseguramos el orden de primera aparición.
    for (char c : texto.toCharArray()) {
        if (frecuencias.get(c) == 1) {
            return c;
        }
    }
    return null;
}
}

```

Consigna 4: Caso Teórico de Borde y Colisiones

1.

```

public class TeoriaColisiones {
    public static void main(String[] args) {
        // Sabemos por teoría que "FB".hashCode() == "Ea".hashCode() == 2236.
        String s1 = "FB";
        String s2 = "Ea";

        Set<String> conjunto = new HashSet<>();

        // Inserción de s1:
        // - Se calcula hash = 2236 -> bucket "X".
        // - Se guarda "FB" en bucket "X".
        conjunto.add(s1);

        // Inserción de s2:
        // - Se calcula hash = 2236 -> bucket "X" (el mismo).
        // - Java verifica si existe algún elemento en bucket "X" que sea .equals(s2).
        // - Recorre la lista: encuentra "FB". Pregunta "FB".equals("Ea") -> false.
        // - Como son diferentes, agrega "Ea" como un NUEVO ELEMENTO en el mismo
        bucket.
        conjunto.add(s2);

        // RESULTADO: El conjunto contiene 2 elementos.
        // La colisión de hash NO reduce el tamaño, siempre y cuando equals() los distinga.
        System.out.println("Cantidad de elementos: " + conjunto.size()); // 2

        // Verificación de existencia:
        System.out.println("Contiene 'FB'? " + conjunto.contains("FB")); // true
    }
}

```

```
        System.out.println("Contiene 'Ea'? " + conjunto.contains("Ea")); // true
    }
}
```

2.

El rendimiento de HashMap (y HashSet) depende de la distribución de los hashCode():

Escenario Ideal ($O(1)$): Los hashes están distribuidos uniformemente. Cada bucket tiene ~ 1 elemento. La búsqueda calcula el hash, va al bucket y devuelve el elemento en tiempo constante.

Escenario con Muchas Colisiones (Peor Caso):

Java 7 y anteriores: Si todas las claves caen en el mismo bucket, la estructura interna degenera en una lista enlazada. La búsqueda requiere recorrer toda la lista $\rightarrow O(n)$.

Java 8+ (Optimización): Si un bucket acumula más de 8 elementos, la lista enlazada se convierte en un árbol rojo-negro (TreeNode). La búsqueda en un árbol balanceado es $O(\log n)$. Por lo tanto, el peor escenario posible en Java moderno es $O(\log n)$, no $O(n)$, lo que mitiga el ataque de colisiones.